IMPLEMENTING AI QUANT TRADING

AI Quant Trading

with Python

HMMs, Kalman Filters, and Statistical Arbitrage

Reinforcement Learning, Backtesting, and Risk Management



Ted Park

Request | Build | Verify | Repeat

Implementing AI Quant Trading with Python

HMMs, Kalman Filters, Statistical Arbitrage, Reinforcement Learning, Backtesting, and Risk Management

Copyright © 2026 Ted Park.

All rights reserved.

First edition, 2026.

Set in Apple SD Gothic Neo | Menlo.

Contents

Preface **1**

How This Chapter Is Organized . . . 1

Learning Goals . . . 2

The Chart That Fooled Me . . . 2

What Technical Analysis Looks Like Today . . . 2

What Renaissance Technologies Figured Out . . . 3

A Different Way of Seeing the Market . . . 3

Why I Wrote This Book . . . 4

What AI Coding Assistants Made Possible . . . 4

How to Read This as a Machine-Learning Trading Book . . . 5

A Map for When You Get Lost . . . 6

A Walk-Through Example . . . 8

Breaking Down the Model More Simply . . . 8

What This Book Covers, and What It Doesn't . . . 9

What You'll Be Able to Build by the End . . . 10

Principles Drawn from the Renaissance Story . . . 10

1. The Causality Principle . . . 11

2. The Ensemble Principle . . . 11

3. The Walk-Forward Validation Principle . . . 11

The Road Ahead . . . 11

A Note on Reading Backtest Results . . . 12

Full Chapter Roadmap . . . 13

Reading Differently on Each Pass . . . 14

Minimum Goal by Part . . . 15

Using AI as a Pair Programmer 16
Key Takeaways from This Chapter 16
Prerequisites 16

Chapter 0. Vibe Coding Quickstart: Verify One Small Example First 18

Establish the expected result first 18
Run the check directly from the text 19
How to ask AI for help 20
Check the order of events 20
Narrow down a failure 20
Before moving on 21

Preface

No separate source files, example ZIP, or repository access are provided. Save the standalone examples in each chapter from the book into local files to run them. Other excerpts, including names such as `app.trading`, illustrate system architecture and are not complete runnable applications.

How This Chapter Is Organized

This preface sets the direction for the book. We'll look at why chart-based judgment hits a wall, and then we'll see how this book builds a quant trading system.

- A time when a chart fooled me
- Why technical analysis struggles in today's market
- The key perspective drawn from Renaissance Technologies
- What this book covers and what it does not
- The system we'll build and what you need to prepare

Learning Goals

After reading this preface, you should be able to explain the following.

- Why this book starts with data pipelines and validation rather than chart patterns
- Why “secretly peeking at future data (Look-ahead Bias)”is fatal in a quant system
- What roles HMM, reinforcement learning, and statistical arbitrage play in this book
- What basic ML terms like training, inference, features, and missing data mean inside a trading system
- How much Python and statistics knowledge you need before reading this book

The Chart That Fooled Me

I once stared at a candlestick chart for a long time and decided it was the right moment to buy. The price had bounced off the same support level three times, RSI was in the oversold zone, and the MACD golden cross looked exactly like the textbook example. Several signals seemed to point the same way, and I trusted them and bought.

The next day the price sliced right through that “perfect”support level. It wasn’t the first time, and it wasn’t the last. But after that day my question shifted a little. Was I simply bad at reading charts, or were the tools I’d been leaning on simply not built for today’s market?

What Technical Analysis Looks Like Today

For the next few months I read papers, browsed trading forums, and looked through performance data, asking the same question over and over. The short answer: the tools I’d been relying on were not enough to explain today’s market.

Traditional technical analysis means tools like candlestick patterns, moving-average crossovers, Bollinger Bands, and RSI. These tools were created in an era when markets moved slowly and human psychology drove most of the price action. Back then they actually worked. Today is different.

In today's market, algorithms place orders far faster than people can, and they account for a large share of trading volume. Patterns that once reflected crowd psychology are now signals that machines detect first and act on first.

When a support level shows up on a chart, humans aren't the only ones who see it. Algorithms see it too. The spot where stop-loss orders pile up is also where liquidity pools, and that zone often gets deliberately tested. A signal that worked well in the old market may not behave the same way in today's market.

I'm not trying to be pessimistic. I'm just pointing out that many of the tools individual investors commonly use were built around a different kind of market than the one we have now.

That's why this book doesn't start with "how to read charts better."Instead, it treats the market as a process that produces data. From there, we move toward estimating the hidden states inside that process and checking whether those estimates accidentally peeked at future data.

What Renaissance Technologies Figured Out

The place that pursued this approach first, and most rigorously, is Renaissance Technologies.

Jim Simons founded Monemetrics in 1978, renamed it Renaissance Technologies in 1982, and launched the Medallion Fund in 1988. Instead of relying only on traders who read stock charts, the firm recruited mathematicians, physicists, and computer scientists to search for statistical patterns in price data. Based on publicly reported results, Medallion is known for an exceptional long-term investment record.

A Different Way of Seeing the Market

Simons didn't have some special inborn intuition. The difference was in how he looked at the market.

Renaissance didn't view the market as something governed by one fixed law. Instead, they saw the market as moving between several "hidden states."A hidden state is a regime that doesn't show itself directly but still controls how the market behaves. In some periods a trend keeps going strongly; in others, prices that went up snap back to the average; in others, volatility suddenly explodes.

You can't see these states directly. But you can estimate them from the data. If you can figure out which state the market is in right now, you can pick a strategy that fits that state.

The HMM (Hidden Markov Model) is the tool that handles exactly this "estimate the invisible state from data"problem. We'll cover it in detail in the middle of the book.

> I'm not an insider at Renaissance Technologies, and I don't know their exact methodology. But I was able to implement part of their philosophy in code by drawing on public interviews and academic research. This book is a record of that process.

Why I Wrote This Book

Quant trading is hard to learn. Not because each individual concept is impossible, but because too many pieces arrive at once. You need statistics, signal processing, Python implementation, a backtest engine, broker integration, and risk management to work together.

Among all of that, one thing is the trickiest. **Building a system that doesn't fool itself.**

There's a trap called "Look-ahead Bias."It's the mistake of secretly using future data while building a strategy. It hides in many places in your code, and it's not easy to spot. Without careful validation, a strategy that peeked at the future can look like it produced amazing results. Returns that beat the market average are usually called "alpha,"but what looks like alpha may just be the result of peeking at future data.

Most people who want to get into quant trading give up before they've built this foundation. Then they fall back to candlestick charts and RSI. Not because those tools are enough, but because they are easy to reach for. I wanted to close that gap. Without hiding the math or the implementation, I wanted to lay the material out in an order a reader can actually follow.

What AI Coding Assistants Made Possible

Since 2023, AI coding assistants have reached a practical level. That's what made this book possible. Tools like GitHub Copilot, Claude, and ChatGPT help explain complex algorithms, find subtle implementation bugs, and unpack what a formula means in code. They don't do the understanding for you. But they sharply cut the time and trial-and-error it takes to get there.

I built the system in this book with AI as a working partner. When I got stuck on difficult math, when I was hunting subtle bugs, when I was picking hyperparameters (the settings that control how well a model performs), AI helped at every step. This book walks through that process from the beginning. The goal is not to stop at theory. It is to show how to build a real system with the tools you can use right now.

How to Read This as a Machine-Learning Trading Book

This book covers HMM and reinforcement learning, but more broadly it is a learning guide for building a stock-trading system with machine learning and deep learning.

The core flow is simple.

```
stock market
  -> data
  -> features
  -> machine-learning model
  -> deep-learning / RL agent
  -> validation
  -> order execution
```

The word **model** can feel vague at first. Intuitively, think of it this way: a model is not a magic box that predicts tomorrow's price. It is a function whose internal numbers were learned from past data. A normal function has rules written directly by a person. A machine-learning model has the same input-output shape, but the internal weights are fitted from data.

```
feature row today -> trained model -> regime probability / action
proposal
```

If the word **model** still feels vague, use this simpler image.

> **model = a decision calculator that has learned its internal numbers from data**

A normal calculator returns 5 when you enter 2 + 3. A trading model receives numbers such as return, volatility, volume, and z-score, then returns a judgment such as "mean-reversion

regime 70%"or "use 35% of the allowed position size."It is not the data itself. It is not just the algorithm name. It is the learned decision function the algorithm produces after training on data.

```
def model(today_features):
    return prediction
```

That is why this book separates three words carefully.

Word	Easy meaning	Example in this book
Algorithm	The method used to build a model	HMM Baum-Welch, SAC training
Parameter	A number learned inside the model	HMM transition matrix, neural-network weights
Model	The learned decision function	today's features -> regime probabilities

As a cooking analogy, data is the ingredient, the algorithm is the recipe, parameters are the heat level and seasoning ratios, and the model is the finished way of cooking after those ratios have been fixed. The same HMM algorithm can produce different models depending on which market period it trained on.

Training means fitting the model on historical data. Inference means freezing that trained model and feeding it new data one row at a time. OOS validation hides later data from the model during training, then checks whether the trained model still works there.

At first, do not try to understand every algorithm name at once. Hold on to this backbone: **data -> features -> training -> inference -> validation -> orders**. HMM, SAC, risk management, and MLflow all attach to that one backbone.

A Map for When You Get Lost

Here is the same idea in trading language.

Machine-learning word	Trading meaning in this book
Data	Historical prices, volume, macro indicators, and broker records
Feature	A number the model can read, such as return, volatility, momentum, z-score
Training	Letting the model fit its internal numbers from past data
Inference	Asking the trained model to judge today's new row
OOS	The exam period hidden from the model during training
Serving	Running the trained model as a service that the trading engine can call
Risk layer	The final guard that can block or shrink the model's proposal

If you get lost later, return to this table. Most of the harder chapters are really just one row of this table explored in more depth. Chapter 2 builds data and features. Chapters 4–5 train and infer HMM regimes. Chapters 13–14 train an RL agent. Chapter 16 limits risk. Chapter 27 serves the trained model.

For the first pass, use this checklist.

```
[ ] Can I explain what data goes into the model?
[ ] Can I explain what feature is created from that data?
[ ] Can I separate training data from inference data?
[ ] Can I point to the OOS region?
[ ] Can I say what the model outputs?
[ ] Can I say what risk check happens before an order?
```

If you can answer those questions, you are following the real structure of the book even if HMM, SAC, Kelly, or CVaR still feel unfamiliar. The details become much easier once the pipeline is clear.

A Walk-Through Example

A concrete example helps. Suppose you have five years of daily data. A beginner-friendly split might train the model on the first two years and run inference on the next three years. The second part is still historical data on your disk, but during validation it must behave like future data. The model should not fit on it. It should only receive one feature row at a time and produce an output.

```
5 years of features

first 2 years: training data
  -> fit scaler
  -> fit model
  -> save model parameters

next 3 years: OOS inference data
  -> load trained model
  -> feed one row at a time
  -> record model output
  -> pass through risk checks
```

Breaking Down the Model More Simply

This is also the easiest way to understand a model file. A model file is not the training data itself. It is the result of training: the learned parameters, the scaler, the mapping rules, and sometimes the last state needed to continue inference. Serving means loading that model file and making it answer requests.

```
training creates the model file
serving loads the model file
inference sends new features into the loaded model
risk decides whether the model output can become an order
```

The same pattern appears throughout the book. HMM training creates regime parameters. HMM inference reads today's regime probability. SAC training creates an actor. SAC inference proposes an action. MLflow records which model file came from which training run. Risk management keeps the model output from becoming an oversized order.

In live trading, training and inference run on two separate cadences.

```
Slow clock: training / validation / model registration
historical data -> training -> OOS validation -> save model file ->
approve

Fast clock: real-time inference / order flow
today's data -> features -> load model file -> inference -> risk check
-> order -> fill check -> monitoring
```

During the trading day, the model is not a student studying again every minute. It is closer to a calculator in the exam room. The learning already happened elsewhere. The live system loads the approved model file, sends today's feature row into it, receives the output, then lets the risk layer decide whether that output can become an order.

A saved model file therefore means more than "one file on disk."It fixes the input format, the learned numbers, and the meaning of the output together. If training used `[return, volatility, momentum, volume]` but inference sends `[return, volume, volatility, momentum]`, the model still receives four numbers, but their meanings are wrong. This is why later chapters keep returning to feature order, scalers, regime label mapping, serving, and production validation.

What This Book Covers, and What It Doesn't

Let's draw the boundary of this book first.

What this book is not

- It's not a Python beginner book. It assumes you can write loops and import libraries.
- It's not a chart-trading manual. It doesn't cover chart-based technical analysis like support lines or MACD divergence.
- It doesn't promise quick riches. Quant trading is hard, and the risk is real.

What this book is

It's a hands-on guide to building a quant trading system from scratch. We'll construct a system in the style used by quant hedge funds like Renaissance Technologies, step by step, with help from AI. The code in this book actually runs, and the results are reproducible numbers.

What You'll Be Able to Build by the End

System	One-line description
HMM market-regime detector	Estimates whether the market is in an up, sideways, or down regime using only the data available so far —no peeking at the future.
Multi-asset HMM	Reads signals from several assets such as stocks and bonds together, for a more accurate regime read.
Two-layer macro/micro HMM	Looks at the macro economy and individual assets in two layers and classifies the market as Risk-On, Neutral, or Risk-Off.
Reinforcement-learning trading agent	Takes the regime detected by the HMM as input and decides positions on its own.
Statistical arbitrage system	Finds pairs of assets that move together and trades the gap between their prices.
Production deployment system	Connects to brokers like IBKR (Interactive Brokers) and KIS (Korea Investment & Securities) to place real orders.

The real goal of this book is to understand why each part is needed, and how it breaks when built wrong.

Principles Drawn from the Renaissance Story

Three principles run through this entire book, all drawn from how Renaissance Technologies approaches trading.

1. The Causality Principle

> Future data must never be used in a current decision.

It sounds obvious, but it is surprisingly easy to violate when you write code. Even widely used library functions sometimes mix future information into the result. In this book, instead of relying on those functions, we directly implement an approach (the Forward Algorithm) that keeps the promise of "only look at data available up to now."Chapter 5 explains the reasoning in detail. Once you understand it, you will start spotting the same kind of problem in other code.

2. The Ensemble Principle

> Don't depend on a single model.

This system uses a daily-bar HMM, an hourly-bar HMM, a reinforcement-learning agent, and a statistical-arbitrage signal together. When the models disagree, we wait. When they all agree, we act.

3. The Walk-Forward Validation Principle

> The only way to confirm a strategy works is to test it repeatedly, in time order, on data the model has never seen.

Splitting the data once into a training set and a test set isn't enough. Walk-forward validation is the baseline standard applied to every strategy in this book.

The Road Ahead

We'll stack the whole system in this order: data -> noise reduction -> regime detection -> position sizing -> validation and live operation. Each step ties to a cluster of chapters.

Step	What we do
1	Collect and store market data
2	Reduce the noise mixed into prices
3	Use a multi-asset HMM to estimate the market regime
4	Sort the regime into Risk-On / Neutral / Risk-Off using two layers, macro and micro
5	Decide how large a position to take based on the regime
6	Look for extra opportunities through statistical arbitrage
7	Have a reinforcement-learning agent help with position decisions
8	Add risk management, broker connection, and monitoring for live operation

On the first read, think of Chapters 1-18 as **one pass through the basic system from end to end**. Chapters 19-27 go deeper into the same material. They cover pair theory, the complete HMM pipeline, production validation, distributional RL, and MLflow serving —the topics you'll come back to when you start actually running the system.

A Note on Reading Backtest Results

This book shows several backtest results, but no single number should be read as a promise of performance. Even the same strategy can produce very different results depending on the data period, trading costs, the universe of symbols, the retraining frequency, and the slippage assumptions.

On your first read, look at these three things rather than the numbers themselves.

- Did it avoid using future data?
- Was maximum drawdown shown alongside trading costs?
- Does the pattern still hold on out-of-sample data?

Full Chapter Roadmap

Part	Chapters	Topic
I	Preface ·1-3	The full system picture, Look-ahead Bias, data pipeline, backtest engine
II	4-5	HMM basics and learning algorithm, inferring without using future information
III	**6-8**	Kalman filter (noise reduction), multi-asset HMM, two-layer macro/micro HMM (the heart of the system)
IV	9-10	Position sizing, making directional signals with an HMM ensemble
V	11-12	Finding pairs that move together and building statistical-arbitrage signals
VI	13-15	Reinforcement-learning environment, SAC agent, hyperparameter tuning
VII	16-18	Risk management, broker integration, real-time monitoring
VIII	19-27	Deep dives: pair theory, regime-based Z-score, full pipeline, production validation, distributional RL, MLflow serving

> Most of the terms in this table will look unfamiliar at first. Don't try to memorize them now. On the first read, take one loop through Parts I-VII to go from "data to orders and monitoring."Treat Part VIII as the deeper section you turn to on a second read, or when you start polishing a real system.

Reading Differently on Each Pass

On the first pass, read for the flow. Do not stop because one equation is unclear. Ask only where the data enters, what the model outputs, and where risk blocks an order. On the second pass, return to the code and formulas. On the third pass, start replacing the examples with your own symbols, costs, and validation periods.

This book is not meant to be fully absorbed in one pass. If you try to understand every parameter and every implementation detail from the beginning, you will lose the main flow. Use a different goal for each pass.

Pass	Goal	How to read
First pass	Grasp the big flow	Watch how data becomes model judgment and then orders
Second pass	Connect code and concepts	Check each chapter's input, output, and failure point in code
Third pass	Experiment and modify	Change parameters and see how OOS results move

On the first pass, it is fine if the formulas are not fully clear. Instead, ask which of the following questions the chapter is answering.

```
Is this chapter preparing data?
Is it reading the market?
Is it finding an opportunity?
Is it deciding size?
Is it blocking risk?
Is it checking operational health?
```

On the second pass, drill into the code. Check where `fit()` is called, where `predict()` or `step_forward()` is allowed to run, and whether the OOS region is kept out of training. At that point, equations stop being something to memorize and become a map explaining why the code is shaped the way it is.

On the third pass, experiment. Change window length, entry threshold, number of regimes, trading costs, and risk limits. But do not keep tuning against the same OOS result. If you change a value, write down the hypothesis behind the change and keep validation and final test periods separate.

Minimum Goal by Part

On the first read, each Part has one sentence you should be able to explain.

Part	Minimum understanding goal
I	Build the foundation so data and backtests do not lie
II	HMM reads market state as probabilities, not future price
III	Noise reduction and multi-market views make regime judgment more stable
IV	A signal gives direction, and Kelly turns that direction into position size
V	Pairs trading trades the relationship between two prices, not the direction of one stock
VI	RL is not a magic AI that creates every signal; it adjusts position size
VII	Even if the model is right, bad risk, orders, or monitoring can break the live system
VIII	Re-check the same material more strictly and shape it into an operational system

Use this table as the first-read standard. If you can explain the minimum goal of each Part, the first pass has done its job even if some formulas still feel difficult.

Using AI as a Pair Programmer

Throughout this book, I'll show directly how I used an AI coding assistant whenever I got stuck. The prompts I used to understand an algorithm, the subtle bugs the AI found, and the moments where AI nudged me toward a better direction are all woven into the text.

The goal isn't to hand over your thinking to the AI. **Treat it like a strong senior colleague.** Ask about what you don't know, confirm what you think you understand, move quickly through parts you already know, and work through the hard parts together.

Quant trading isn't a domain reserved for PhDs alone. With the right tools and direction, you can build a practical trading system yourself. This book is a record of doing exactly that.

Key Takeaways from This Chapter

- This book isn't about memorizing chart patterns better. It's about building a system that sees the market through data.
- The single most important promise is **"don't use future data in a current decision."** If that promise breaks, every impressive-looking result is fake.
- Don't stake everything on a single model. Act only when several models point the same way.
- You don't need to master all the math up front. Each chapter walks you through "why this concept is needed"and "what goes wrong if you build it incorrectly,"in order.

Prerequisites

Before we dive in, you should have at least the following ready.

- **Python 3.12 or higher**
- **Basic NumPy and Pandas experience** —enough to handle arrays and manipulate dataframes.

- **Basic statistics concepts** —knowing what mean, variance, and the normal distribution are is enough. Harder concepts are explained inside the book when they're needed.
- **Curiosity about why the market moves the way it does**

If you're ready, the next chapter rolls out the blueprint for the entire system.

Chapter 0. Vibe Coding Quickstart: Verify One Small Example First

The goal of this chapter is to **check whether the profit and loss explained by AI is correct**, using a small numerical example. You do not need the author's production repository, a broker account, or market data. If you have Python, run the code below. Otherwise, calculate the table by hand.

Establish the expected result first

Start with 1,000 in cash and buy 10 units of an asset priced at 100. The price then falls to 80 and returns to 100. Ignore costs for this first calculation.

Point in time	Cash	Quantity	Price	Equity
Before the trade	1,000	0	100	1,000
After buying	0	10	100	1,000
After the price falls	0	10	80	800
After the price recovers	0	10	100	1,000
After closing the position	1,000	0	100	1,000

Equity is `cash + quantity × price`. The final profit is zero, but the maximum drawdown is `(1000−800)/1000=0.2`, or 20%. Looking only at the final balance hides the risk taken along the way.

Run the check directly from the text

Paste this into a new file named `first_check.py`. It uses only Python's standard library; no downloads are required.

```
prices = [100.0, 80.0, 100.0]
initial = 1000.0
quantity = initial / prices[0]
cash = initial - quantity * prices[0]
equity = [initial] + [cash + quantity * p for p in prices]
peak = initial
mdd = 0.0
for value in equity:
    peak = max(peak, value)
    mdd = max(mdd, (peak - value) / peak)
final_cash = cash + quantity * prices[-1]
assert final_cash == 1000.0
assert mdd == 0.2
print(final_cash, mdd)
```

Run `python3 first_check.py` in a terminal. The output is `1000.0 0.2`. On Windows, use `py first_check.py` if your Python command is `py`. This checks profit and loss for a fixed quantity. It does not generate signals, place real orders, or train an HMM.

How to ask AI for help

```
Review the table and Python code together.
1. Calculate equity and maximum drawdown by hand.
2. Explain what risk is missed if only cash is recorded.
3. Calculate the final cash if buying and selling each incur a 10 bp
fee.
4. Do not add a broker, external data, or new packages.
After answering, identify the table values you used to verify your
answer.
```

Ten basis points (10 bp) is 0.1% of the trade value. Buying and selling 10 units at 100 costs 1 each time, or 2 in total, leaving 998 in cash. Because this example uses the entire initial balance to determine quantity, cash temporarily becomes negative by the purchase fee of 1. An actual account that forbids borrowing must buy fewer units to leave room for the fee. Distinguish a fixed-quantity exercise from real account constraints.

Check the order of events

A signal based on today's closing price cannot be filled at today's earlier opening price. The book's basic exercise uses **finalize the signal after bar t closes -> fill at bar t+1's open -> value the position at bar t+1's close**. It is valid to include that day's profit or loss for a position opened using a previously finalized signal.

Narrow down a failure

Symptom	First check
Python command not found	Check whether the installed command is python3 or py
Indentation error	Check that you copied only the code and preserved its indentation
Drawdown is zero	Check that equity of 800 at a price of 80 was recorded
Final cash and total profit do not reconcile	Check for profit added twice or duplicated costs

Compare the failing lines with the example before installing more packages. Prioritize agreement between the expected result and actual output over how convincing the generated answer sounds.

Before moving on

- Explain why zero final profit and a 20% maximum drawdown can both be correct.
- Charge each fee once per actual fill.
- Distinguish signal generation time from execution time.

Once these checks make sense, continue to the architecture in Chapter 1. No separate example files are provided; save the code printed in the book to run the exercises. Names such as `app.trading` describe examples from the author's system architecture. Understanding the concepts does not require access to that private package.

Continue with the full book

This short preview ends here. The full edition contains the following chapters.

- Preface
- Chapter 0. Vibe Coding Quickstart: Verify One Small Example First
- Chapter 1. System Architecture —The Design Philosophy of an AI Quant Trading System
- Chapter 2. The Data Pipeline —DuckDB and Live Feeds
- Chapter 3. The Backtest Engine —A Simulation You Can Trust
- Chapter 4. HMM Basics —Inferring Market Regimes with Probability
- Chapter 5. HMM Forward-Only —Real-Time Inference Without Future Information
- Chapter 6. The Kalman Filter —Denoising and the Innovation Z-Score
- Chapter 7. Cross-Asset Joint HMM —Regime Detection Across Multiple Markets
- Chapter 8. Two-Layer HMM Ensemble —Layer A × Layer B
- Chapter 9. Half-Kelly Position Sizing
- Chapter 10. HMM Forward Ensemble —V1 -> V3 Evolution
- Chapter 11. Statistical Arbitrage —Pair Selection

- Chapter 12. Statistical Arbitrage —Generating Signals
- Chapter 13. Designing the RL Environment —StatPairRLEnv
- Chapter 14. The SAC Agent —Architecture and Training
- Chapter 15. Optuna Hyperparameter Tuning
- Chapter 16. Risk Management —Protecting Positions and Measuring Performance
- Chapter 17. Connecting to IBKR —Real-Time Order Execution
- Chapter 18. Monitoring —Real-Time Performance Tracking and Alerts
- Chapter 19. Bamberger Statistical Pairs Arbitrage —Theory and Principles
- Chapter 20. Pair Selection —Regime-Conditional Filtering
- Chapter 21. Regime-Based Z-Score —The Core Strategy Logic
- Chapter 22. The HMM Regime Strategy —A Complete Pipeline From Data to Signal
- Chapter 23. SAC RL Agent Preview —An Introduction to Reinforcement Learning
- Chapter 24. Production Validation
- Chapter 25. Comparing RL Algorithms —SAC vs PPO vs QR-DQN
- Chapter 26. Distributional RL and CVaR —Trading the Tail Risk
- Chapter 27. MLflow Model Registry and Serving —Production Deployment
- Appendix. Beginner's Map

Get the full edition